

Práctica 5

1) $\Gamma_1(A); w_1(A); \Gamma_2(A); w_2(A); \Gamma_1(B); w_1(B); \Gamma_2(B); w_2(B)$
 $\Gamma_1(A); w_1(A); \Gamma_1(B); w_1(B); \Gamma_2(A); w_2(A); \Gamma_2(B); w_2(B)$

2) H_1 : RC, dirty read, lost update

H_2 : RC, ACA, lost update

H_3 : RC, ACA, lost update

H_4 : RC, dirty read, lost update

H_5 : RC, ACA, lost update

3) H_1 : RC, ACA, ST

H_2 : NO-RC

H_3 : RC, ACA,

4) $T_1 T_2 C_2 C_1$

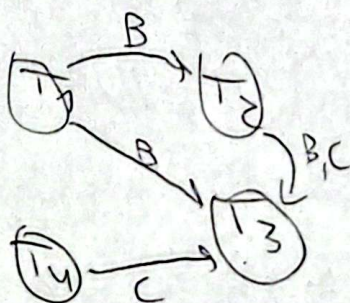
$\Gamma_2(D); \Gamma_2(B); w_1(B); \Gamma_1(C); w_2(C); w_1(C)$
 $C_1; C_2$

2.1) $l_1(A); \Gamma_1(A); w_1(A); U_1(A); l_2(A); \Gamma_2(A); w_2(A); U_2(A);$
 $l_1(B); \Gamma_1(B); w_1(B); U_1(B); l_2(B); \Gamma_2(B); w_2(B); U_2(B)$
 $C_1; C_2$

2.2) $l_1(A); \Gamma_1(A); w_1(A); U_1(A); l_2(A); \Gamma_2(A); w_2(A); U_2(A);$
 $l_2(B); \Gamma_2(B); w_2(B); U_2(B); l_1(B); \Gamma_1(B); w_1(B); U_1(B); C_2; C_1$

2.3) $l_1(A); A = A + 2; l_1(B); B = A + 5; U_1(B); U_1(A); l_2(A);$
 $A = A + 1; l_2(E); E = A + 8; l_2(D); l_1(C); C = 2C; U_1(C);$
 $U_2(E); D = D / 5; U_2(A); U_2(D); C_1; C_2$

2.3



Una serialización es
 $T_1; T_2; T_4; T_3$

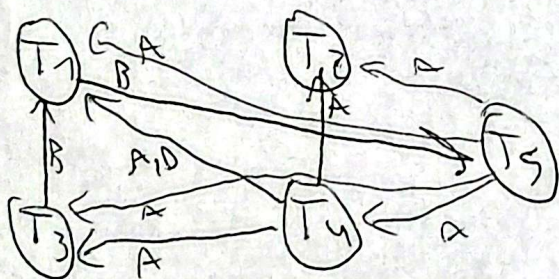
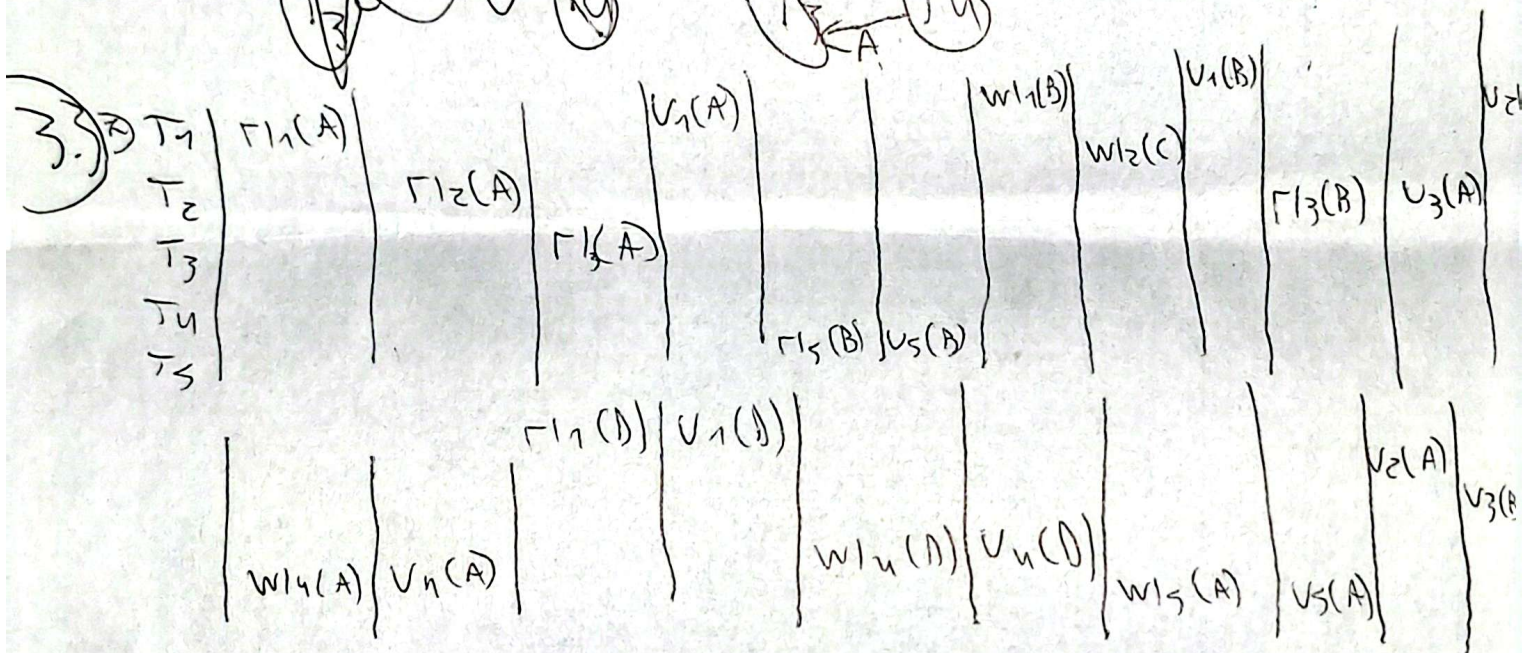
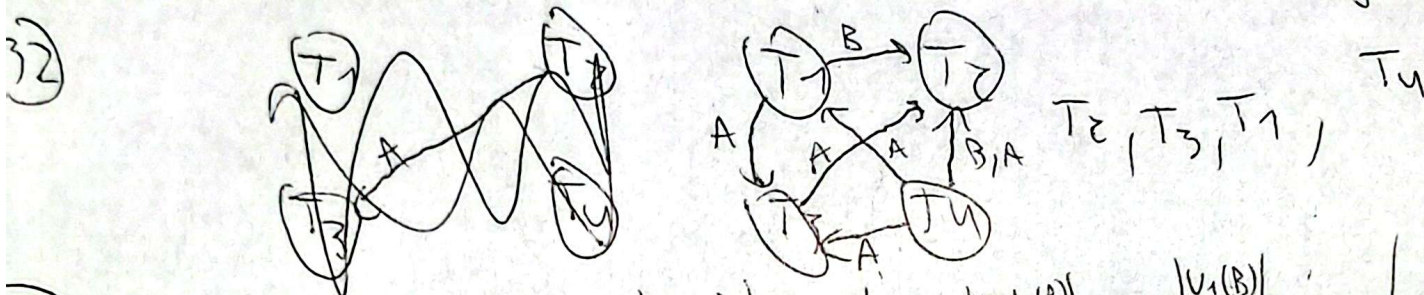
2.4) $l_1(A); \Gamma_1(A); w_1(A); l_1(B); \Gamma_1(B); w_1(B); U_1(A); l_2(A);$
 $U_1(B); l_2(B); \Gamma_2(A); w_1(A); \Gamma_1(B); w_1(B); U_1(A); U_2(B);$

2) a) los ejemplos, primero tomar lock y después lo liberan.

b) $l_1(A); l_2(B)$ es deadlock, T_1 no obtendrá $l_1(B)$ ~ T_2 obtendrá $l_2(A)$

3.1) a) $wl_1(A); A = A + 1; wl_1(B); B = A + B; v_1(A); r_1(A); wl_2(C); C = A + 1; wl_2(D); D = 1; v_2(A); v_1(B); v_2(D); v_2(C); C_2; C_1$

b) si, tienen todos los lock antes de liberar alguno



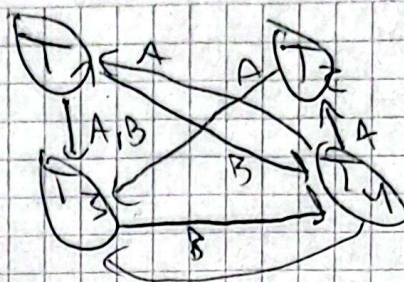
no es serializable, no tiene orden topológico

c) no lo son, menor locks

d) 2PL estricto si libera locks de escritura luego del commit (o abort).

2PL riguroso si libera locks de todo tipo luego del commit (o abort)

3.4) $T_1 T_2 T_3 T_4$



no es serializable

2) T_1 simple, $T_2 = \Gamma_2(A); \Gamma_2(B); U_2(A); U_2(B)$
 $T_3 = W_3(A); W_3(B); U_3(A); U_3(B)$
 $T_4 = \Gamma_4(B); W_4(A); U_4(B); U_4(A)$

3.5)

T_1

$\Gamma_1(A)$

$\Gamma_1(B)$
 $U_1(B)$

$U_1(A)$

$\Gamma_1(C)$

$U_1(C)$

~~T_2~~

~~$W_2(B)$~~

$\Gamma_2(C)$

$W_2(A)$
 ~~$U_2(B)$~~
 ~~$U_2(A)$~~

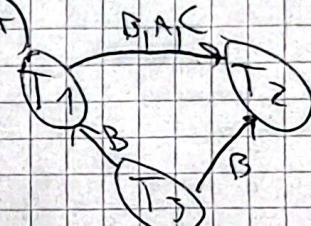
T_3
 $W_3(B)$
 $\Gamma_3(B)$
 $\Gamma_3(C)$
 $U_3(B)$

$U_3(C)$
 $U_3(A)$

$\Gamma_3(D)$
 $U_3(D)$

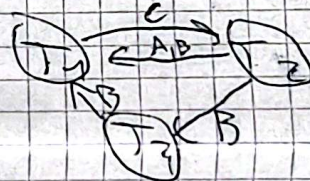
$W_2(C)$
 $U_2(C)$

2)



T_3, T_1, T_2

3) basta con poner T_2 antes de T_3



4) Si, T_3 hace $W_3(B)$ y luego T_1 $\Gamma_1(B)$ antes del commit de T_3

3.6)

T_1

T_2

T_3

T_4

$\Gamma_2(X)$

$\Gamma_3(X)$

$W_3(Y)$
 $U_3(X)$
 $U_3(Y)$

$\Gamma_4(Y)$

$\Gamma_1(Y)$

$U_1(Y)$

$W_1(X)$

$U_1(X)$

$U_1(Y)$

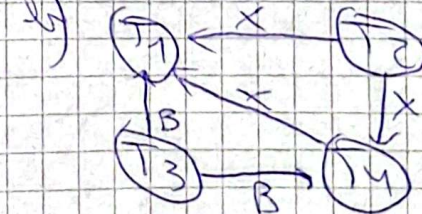
$U_2(X)$

$\Gamma_4(X)$

$U_4(X)$

$U_4(Y)$

a) es legal, T_1 no en CP

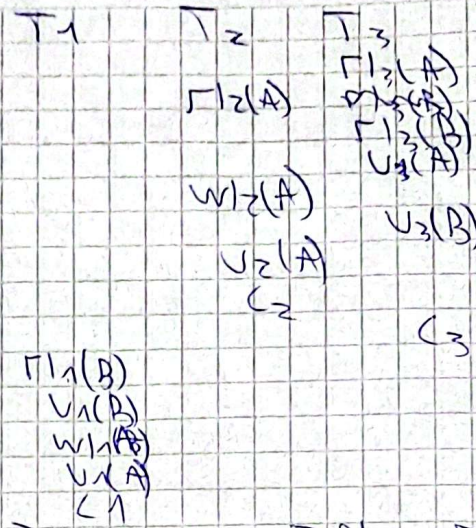
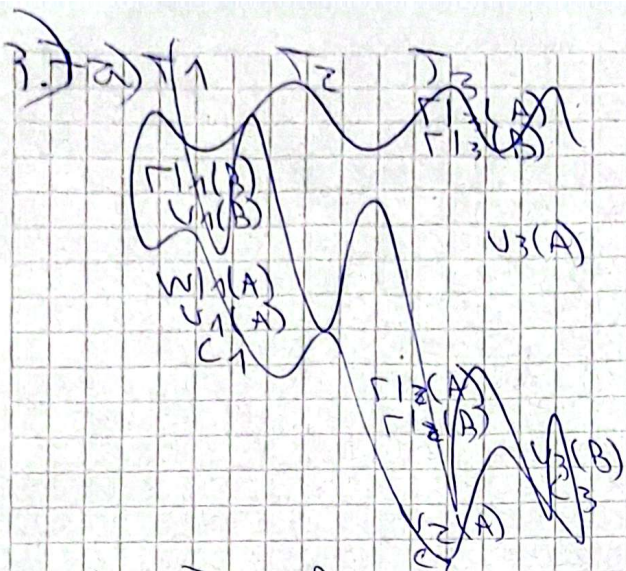


T_2, T_3, T_4, T_1
 T_3, T_2, T_4, T_1

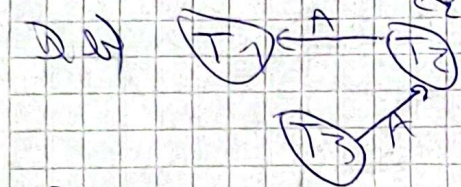
2) RC

3) no RC

4) marcado de interpretacion que se puede un tiempo mas de ser necesario



2) ya en ZPL siguientes



3.8) a) $H_1 = \{r_1(A), r_2(A), r_2(A), r_1(A), r_1(B), r_1(B), r_2(B), r_2(B), u_2(A), u_2(B), c_2, w_1(A), w_1(B), u_1(A), u_1(B), c_1\}$

b) $H_2 = \{r_2(A), r_2(A), r_1(A), r_1(A), r_2(B), u_2(B), r_2(B), r_1(B), r_1(B), u_1(B)\}$

denegado, debe esperar.

3.9) a) $\{r_1(A), r_1(A), T_2, w_1(A), w_1(A), u_1(A)\}$



En un punto que en lugar de $r_1(A)$, pedimos $u_1(A)$ y $u_2(A)$ y entonces se quedará esperando.

3.10) a) $H_1 = \{r_1(A), r_1(A), u_1(A), r_2(B), r_2(B), u_2(B), r_3(C), r_3(C), u_3(C)\}$

etc, como ya liberó lock luego de operación

b) $H_1 = \{r_1(A), r_1(A), r_2(B), r_2(B), r_3(C), r_3(C), r_1(B), r_1(B), r_2(C), r_2(C), r_3(C), r_3(C), w_1(A), u_1(B), w_2(B), u_2(C), w_3(C), w_1(A), w_2(B), w_3(C), u_1(A), u_2(B), u_3(C)\}$

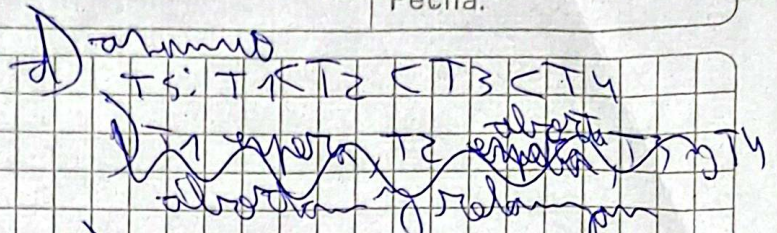
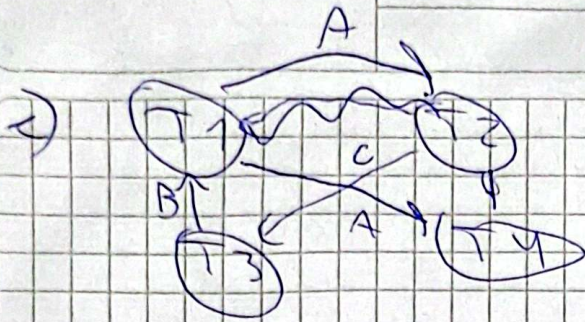
La ejecución depende fuertemente de que se liberen los lock cuando digo, como los transacciones que abortarían por el no poder obtener los lock.

Siempre que el VI del Γ de los que van a tratar de escribir luego, algo habrá diferido

3.11) a) $T_1 = \{r_1(A), r_1(A), w_1(B), w_1(B), u_1(A), u_1(B)\}$
 $T_2 = \{r_2(C), r_2(C), w_2(A), w_2(A), u_2(C), u_2(A)\}$
 $T_3 = \{r_3(B), r_3(B), w_3(C), w_3(C), u_3(B), u_3(C)\}$
 $T_4 = \{r_4(D), r_4(D), w_4(A), w_4(A), u_4(C), u_4(A)\}$

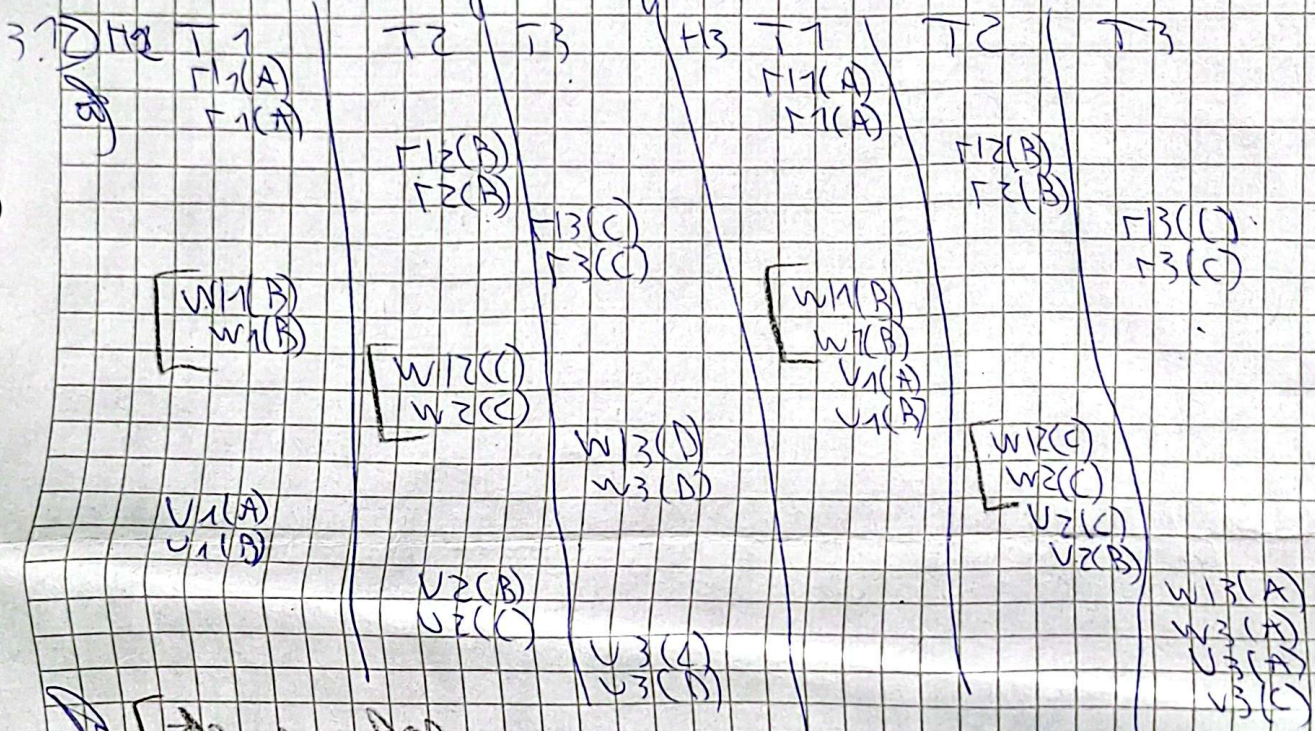
b) $\{r_1(A), r_1(A), r_2(C), r_2(C), r_3(B), r_3(B), r_4(D), r_4(D)\}$

NOTA

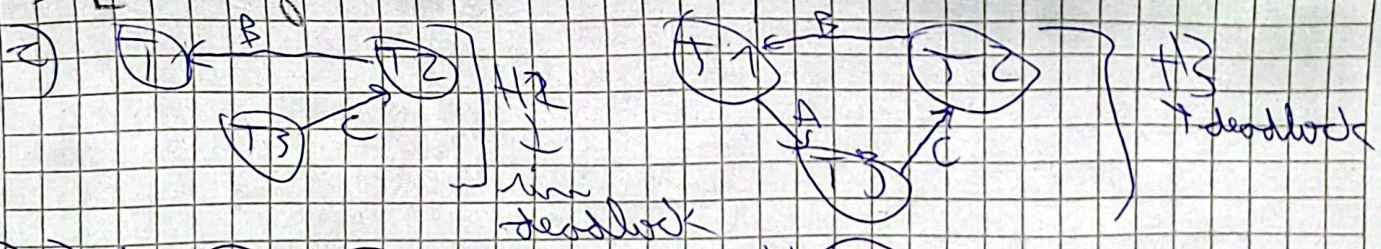


(ii) T1 error, T2 aborta y T3 puede ejecutar y T4 aborta

i) T2 error, T3 error, T4 error, T1 aborta y relanza



[deadlock]



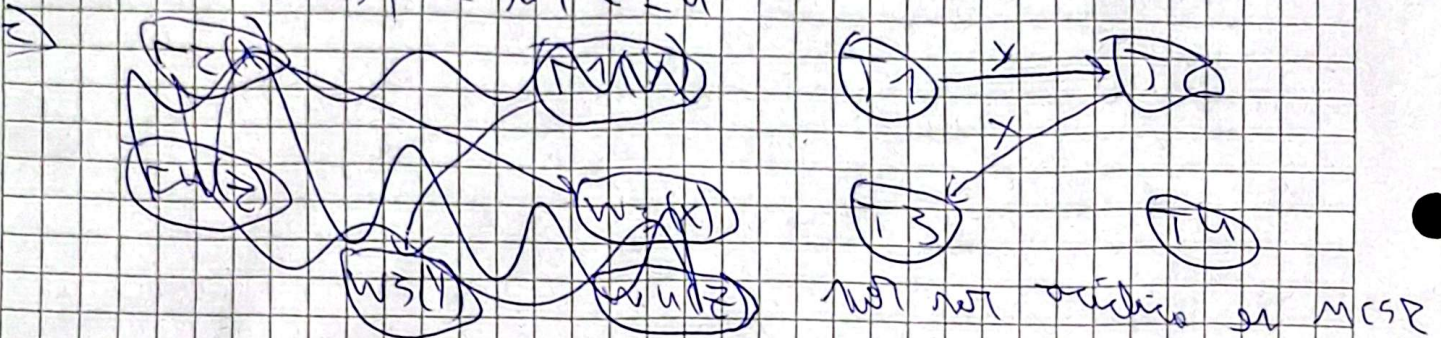
i) H2: todas esperaron
 ii) H2: todas esperaron
 iii) H2: T2 aborta,
 iv) a) H1: el ultimo ~~write~~ write es un write too late, se hace rollback de T1
 H2: igual a H1
 H3: w2(B) es un write too late, aborta T2

igual al anterior, no previene write too late

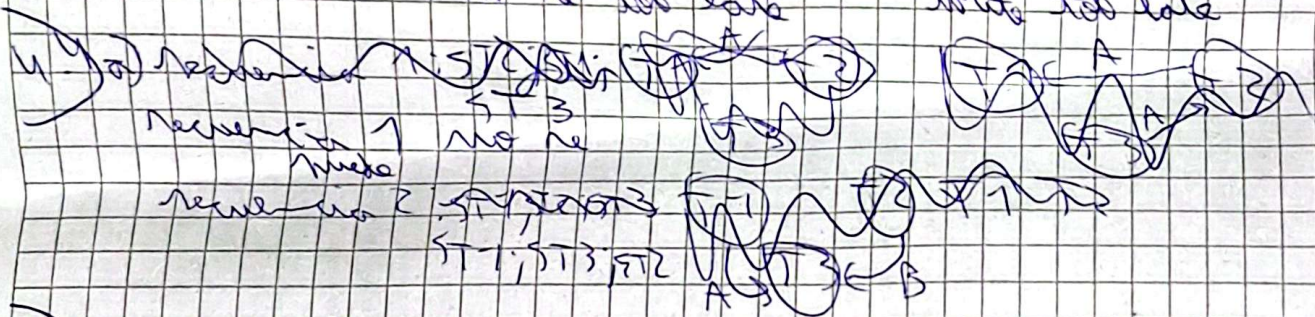
4.2 a) $w_2(x)$ se ^{no se opera} ~~aguarda~~, puesto que se escribió algún anterior, el signal que es $w_1(x)$, $r_3(z)$ aborta. Pero entonces se hacen ~~efectivos~~ $w_2(x)$ y $w_1(x)$

queda $x=2$, $y=1$, $z=4$

igual al anterior pero al hacer $r_3(z)$, obtiene el valor de z original, queda no hay rollback
 $x=3$, $y=30$, $z=4$



4.3 $ST1, ST2, ST3, r1(A), r1(B), r1(B), r2(C), r2(B), r2(A), r3(B), r3(x), w1(x), w1(B), w2(A), w2(B), w3(A), w3(B)$
write too late write too late



4.4 $r1(A), r1(C), r2(C), w3(A), r2(B), w2(B), r3(B), w2(A)$

debe esperar a que termine $T3$, no puede hacer del siguiente orden

4.5 a) $H1$ no es permitida, $r2(x)$ se luego de $w1(x)$, read too late
 $H2$ es permitida

4.6 $H1$ tiene 3 de z por lo que 3 debe esperar en caso final a que z termine para terminar

4.7 $ST2, r2(z), r2(y), ST3, r3(x), w2(y)$
write too late, rollback

4.6) ST 1: $\Gamma_1(A); \Gamma_1(B); w_1(A);$ ~~ST 2~~; ST 3: $\Gamma_3(C); \Gamma_3(B);$ ST 2:
 $\Gamma_2(B); w_3(B); \Gamma_2(C); w_2(A)$

write two sets

6.7) ~~ST 1~~